Deraj Rathore
2023190

End-Semester Examination
Computer Organisation (CSE-112)

Duration = 120 Minutes

Total Marks = 80

INSTRUCTIONS:

1. **This question paper contains four questions. Read all the questions carefully before attempting.**
2. **Electronic gadgets, such as smartwatches, phones, etc., are strictly prohibited.**
3. **In case of assumptions, write it clearly: only the valid assumptions will be considered.**
4. **The use of non-programmable calculators is allowed.**
5. **The institute's plagiarism policy will apply in case any unfair means are observed.**

ISA to be used for this question paper:

- This ISA supports 16 registers (R0-R15).

(a) Basic instructions:

S. No.	Instruction (Mnemonic, Operands)	Operation Performed
1	MUL R1, R2, R3	Multiply the contents of registers R2 and R3, and store the result in register R1.
2.	SUBI R1, #imm	Subtract the content of the specified register (R1) by immediate value (imm) and store the result back in register R1.
3.	BEQ R1, R2, Address/Label	Branch to address/label if the content of register R1 is equal to the content of register R2.
4.	ADD R1, R2, R3	Add contents of registers R2 and R3 and store the result in R1.
5.	MVI R1, #imm	Copy the Immediate value (imm) into the register R1.
6.	ST R1, Address/Label	Store the content of register R1 to the specified address/label.
7.	MOV R1, R2	Copy the content of the register R2 into the register R1.
8.	LD R1, Address/Label	Load the content specified at the mentioned address/label into the register R1.
9.	BL Address/Label	Branch to the mentioned address and store the return address (address of next instruction) in the Link register.
10.	B Address/Label	Unconditional branch to specified address/label.

(b) Some additional instructions/subroutines:

S. No.	Subroutine (Mnemonic, Operand)	Operation Performed
1.	POP R1	Pops the data stored on the top of the stack into register R1.
2.	PUSH R1	Pushes the data stored in register R1 onto the stack.
3.	IN R1	Reads immediate data from the user into register R1.
4.	OUT "str"	Prints string "**str**" on the console.

Q1. [8+12 = 20 Marks]

For this question, register "**R7**" is link register and "**R10**" stores the program counter (PC) value.

(a) Using the ISA provided, write a function called "**factorial**" in assembly code that takes a number as an input from the user and calculates its factorial value. Final result should be stored in the register "R3". You have to ensure that as soon as the calculation of factorial value is finished, the code automatically returns to the called function.

(b) Write assembly code for the "**main**" function which calls the function "**factorial**" defined by you part (a), to use the calculated value of factorial to successively print the string " * " on the console in the following manner example shown for calculated factorial value = 6:

```
******
*****
****
***
**
*
```

⇒ Note that printing " **\n** " moves the cursor to the next line.

Q2. [12+3 = 15 Marks]

Consider a **system A with a cache** and another **system B without a cache**. Both systems have main memory. Details regarding both systems are provided in the table below.

Specifications	System A	System B
Clock	100 MHz	200MHz
Cache Present (access time)	Yes (20ns)	No
Main memory access time	80 ns	80ns

Executing each instruction on System A takes 5 cycles in addition to memory access time. For System B, each instruction takes 2 cycles in addition to its memory access time. None of the systems have pipeline.

(a) An assembly code snippet for a certain application is provided below. The code is executed on systems A and B. For System A, all the memory reads and writes are limited to the cache memory. Calculate the total time it takes for systems A and B to execute this application. Provide the time for both systems in the units of the number of cycles and in nanoseconds (ns).

Note: Line numbers are added just for visual clarity and do not convey any other information.

```
1. ADD R1, R2, R3
2. SUBI R3, #1
3. LD R5, memaddr1
4. SUBI R8, #5
5. ST R8, memaddr2
```

(b) Consider an image processing application with significant temporal locality (requiring frequent reuse of data). Which System (A or B) would you choose to execute such an application, and why?

Q3. [15+5 = 20 Marks]

Consider the following program with five pipeline stages: Fetch (F), Decode (D), Execute (E), Memory (M), and Writeback (W). Each pipeline stage is one clock cycle long. The stages are described below:

1. Fetch - Fetches the instructions from instruction memory
2. Decode - Decodes the instructions and type of operations. It also reads the input operands. If the input operands are not present, the decode stage is stalled for one cycle to read the operands after they are made available.
3. Execute - Executes the instruction in the ALU
4. Memory- For load operation, takes the data from memory [redacted]
5. Writeback - Updates the register value at the end of the stage.

```
1. LD R1,12
2. LD R2,16
3. ADD R11,R2,R3
4. LD R1,4
5. ADD R2,R3,R4
```

```
6.  MUL R5,R1,R5
7.  LD R6,8
8.  LD R7,9
9.  LD R9,9
10. SUBI R6,#3
11. ADD R10,R7,R6
12. ADD R11,R1,R2
```

(a) Draw the pipeline diagram and determine the number of clock cycles needed to execute the above program.

(b) We now add a direct mapped data cache with 8 locations. The structure with locations of the data cache is given below. When the load instruction is executed, the data copied to the register is also copied to the data cache. For the above program, determine the memory addresses present in the data cache after each load instruction. Indicate replacements by striking out the old values and entering new ones. (**Hint**: In a direct mapped cache, each main memory address maps to exactly one unique cache location.)

0	
1	
2	
3	
4	
5	
6	
7	

Q4. [10+10+5 = 25 Marks]

Consider a stack that has the initial value of stack pointer (SP) equal to the memory location **0x45** (hexadecimal). Whenever a value is put into the stack via **PUSH** instruction, the value of the Stack Pointer (SP) **decrements** by one (binary). Whenever a value is moved out of the stack via **POP** instruction, the value of SP **increments** by one (binary). Study the code given below and answer the questions that follow (line numbers added just for visual clarity only):

```
1.  PUSH R2
2.  POP R3
3.  PUSH R5
4.  PUSH R6
5.  PUSH R7
6.  PUSH R8
7.  POP R9
8.  PUSH R1
9.  PUSH R11
10. POP R2
11. POP R3
12. PUSH R14
13. POP R4
14. POP R9
15. PUSH R4
16. POP R3
17. POP R4
18. PUSH R9
19. POP R6
20. POP R1
```

(a) For the given code, determine the value of the updated SP after every instruction is executed.

(b) For the same code, write the updated value of the register modified after every 'POP' instruction, with initial values of R1=1, R2=2, R3=3, R4=4, R5=5, R6=6, R7=7, R8=8, R9=9, R10=10, R11=11 and R14=14.

(c) Also, report only the value pushed in the stack after every PUSH instruction.

Note: You may answer all the above parts (a), (b), and (c), using a common table as shown:

Instructions	PUSHed Value	Register and its value after 'POP'	Updated SP Value